

Appies Legobib

Realisatiedocument

THOMAS
MORE

Bram Van Eccelpoel

2025 - 2026

Inhoudstafel

1. INLEIDING	4
2. ANALYSE.....	5
2.1. GEBRUIKTE TOOLS.....	5
2.1.1. Github	5
2.1.2. Docker.....	5
2.1.3. Dokploy	5
2.1.4. Visual Studio Code	6
2.1.5. Figma	6
2.1.6. StarUML	6
2.1.7. PostgreSQL	6
2.1.8. Zitadel	6
2.1.9. Rebrickable API	7
2.1.10. AG Charts	7
2.1.11. Angular	7
2.1.12. Ngx-translate	7
2.2. PACKAGES	8
2.2.1. Excel Data Reader -> ClosedXML	8
2.2.2. QRCoder.....	8
2.2.3. Tailwind	8
2.3. GEBRUIKTE STRUCTUUR.....	9
2.3.1. Domain-Driven Design (DDD)	9
2.3.2. Vertical slice architecture.....	10
2.3.3. Directives (Angular)	11
2.3.4. Github issues	11
2.3.5. Geautomatiseerde ci/cd.....	12
2.3.6. Signals (Angular)	12
2.4. GEBRUIKERS.....	13
2.4.1. Kinderen en Ouders (Reservatie-/Uitleenportaal)	13
2.4.2. Ziekenhuispersoneel (Bibliotheekportaal)	14
2.4.3. Bricks and More (Beheerders, Managementportaal)	15
3. REALISATIE	16
3.1. ONTWERP VAN DE DATABANKSTRUCTUUR	16

3.2.	INTEGRATIE VAN DE REBRICKABLE API.....	17
3.3.	IMPLEMENTATIE VAN DOMAIN-DRIVEN DESIGN.....	18
3.4.	BEHEER VAN DE BIBLIOTHEEKINFRASTRUCTUUR	19
3.5.	IMPLEMENTATIE VAN ZITADEL EN BEVEILIGING	20
3.6.	ONTWIKKELING VAN HET RESERVATIEPORTAAL	22
3.7.	BEVEILIGING VAN API-ENDPOINTS	23
3.8.	OPTIMALISATIE VAN DE QR-CODE FUNCTIONALITEIT	23
3.9.	IMPLEMENTATIE VAN STATISTIEKPAGINA'S	24
3.10.	IMPORTFUNCTIONALITEIT VIA EXCEL-BESTANDEN	25
3.11.	BEHEER EN WEERGAVE VAN COLLECTIES.....	25
3.12.	EXPORTFUNCTIONALITEIT NAAR EXCEL-BESTANDEN	26
3.13.	IMPLEMENTATIE VAN MEERDERE TALEN	26
3.14.	ONTWIKKELING VAN DE LANDINGSPAGINA	27
4.	BESLUIT.....	28
	GEBRUIK VAN ARTIFICIAL INTELLIGENCE (AI).....	29
	LITERATUURLIJST	30
4.1.	GESPROKEN BRONNEN	30
4.2.	GESCHREVEN BRONNEN	30
	BIJLAGEN.....	31

1. Inleiding

Dit verslag zet systematisch uiteen welke tools en methodieken werden ingezet tijdens de stageperiode, hoe de architectuur van het project tot stand kwam en welke concrete stappen leidden tot het eindresultaat. Het project richtte zich op de ontwikkeling van een innovatieve softwareoplossing voor het beheer en de uitleen van LEGO-sets binnen een ziekenhuiscontext.

De applicatie is ontworpen om gebruikt te worden in meerdere ziekenhuizen en wordt centraal beheerd door Bricks and More. Momenteel gebeurt het beheer van LEGO-sets en uitleenprocessen vaak nog via papieren lijsten of Excel-bestanden. Deze werkwijze is niet optimaal: ze is foutgevoelig, tijdrovend en maakt het moeilijk om snel inzicht te krijgen in de actuele stand van zaken. De nieuwe digitale oplossing biedt een aanzienlijke meerwaarde door processen te automatiseren, het overzicht te verbeteren en de administratieve last voor zowel ziekenhuizen als beheerders te verlagen.

Dit document schetst een helder en volledig beeld van het ontwikkelproces, de technische uitdagingen en de toegepaste oplossingen. Hiermee wordt aangetoond dat het doel, het zelfstandig realiseren van een complex softwareproject volgens de geldende best practices, is bereikt. Voor extra informatie levert het projectplan een diepere kijk op de initiële doelstellingen en planning.

2. Analyse

Tijdens de stageperiode zijn diverse tools en structuren ingezet om het project succesvol te realiseren. Dit hoofdstuk beschrijft de gebruikte hulpmiddelen en motiveert de gemaakte keuzes.

2.1. Gebruikte tools

Onderstaande tools vormden de basis van de ontwikkelomgeving. Voor een overzicht van de documentatie en officiële websites wordt verwezen naar de bibliografie achteraan dit document.

2.1.1. Github

Github werd gebruikt als centrale code repository en versiebeheersysteem. Issues werden ingezet voor het opvolgen van taken, bugs en feature requests, wat de samenwerking binnen het team sterk verbeterde. Dankzij Github Actions werd geautomatiseerde CI/CD opgezet, waardoor code automatisch werd getest en gedeployed bij elke wijziging. Dit zorgde voor een betrouwbare en snelle ontwikkelcyclus. (Github, n.d.)

2.1.2. Docker

Docker werd ingezet voor het containeriseren van applicaties en het vereenvoudigen van de uitrol en het beheer van de verschillende services binnen het project. Door gebruik te maken van Docker konden ontwikkel-, test- en productieomgevingen eenvoudig en consistent worden opgezet. Dit minimaliseerde “works on my machine”-problemen en versnelde onboarding van nieuwe ontwikkelaars. (Docker, n.d.)

2.1.3. Dokploy

Dokploy werd gebruikt voor het eenvoudig en veilig deployen van de applicatie naar de cloud. Dankzij de integratie met Github konden nieuwe versies automatisch uitgerold worden na goedkeuring, wat het releaseproces versnelt en minder foutgevoelig maakt. (Dokploy, n.d.)

2.1.4. Visual Studio Code

Visual Studio Code diende als primaire ontwikkelomgeving. Deze editor biedt uitgebreide ondersteuning voor diverse programmeertalen, integratie met versiebeheer en een breed scala aan extensies die het ontwikkelproces versnellen en verbeteren. Dankzij de ingebouwde debugging en live share functionaliteit werd pair programming en code review eenvoudiger. (Visual Studio Code, n.d.)

2.1.5. Figma

Figma werd gebruikt voor het ontwerpen van gebruikersinterfaces en het uitwerken van wireframes en prototypes. Dit zorgde ervoor dat het team snel feedback kon verzamelen en de UI kon afstemmen op de noden van de eindgebruikers. (Figma, n.d.)

2.1.6. StarUML

StarUML werd ingezet voor het modelleren van de softwarearchitectuur en het opstellen van UML-diagrammen. Dit hielp bij het visualiseren van de structuur en de onderlinge relaties binnen het systeem, wat essentieel was voor het afstemmen van backend en frontend. (StarUML, n.d.)

2.1.7. PostgreSQL

PostgreSQL fungeerde als relationeel databasesysteem voor het opslaan en beheren van projectdata. De keuze voor PostgreSQL is gebaseerd op de betrouwbaarheid, schaalbaarheid en de uitgebreide ondersteuning voor complexe datamodellen, wat belangrijk was voor het beheren van de verschillende entiteiten (zoals Lego-sets, reservaties, gebruikers, ...). (PostgreSQL, n.d.)

2.1.8. Zitadel

Zitadel werd gebruikt als identity- en access management-oplossing. Hiermee werd authenticatie en autorisatie op een veilige en schaalbare manier gerealiseerd. De keuze voor Zitadel werd mede bepaald door de bestaande expertise binnen dotNET lab en de vlotte integratie met .NET en Angular. (Zitadel, n.d.)

2.1.9. Rebrickable API

De Rebrickable API werd geïntegreerd om gegevens over Lego-sets op te halen. Dit maakte het mogelijk om actuele en gedetailleerde informatie te tonen binnen de applicatie, wat de gebruikerservaring aanzienlijk verbeterde. (Rebrickable, n.d.)

2.1.10. AG Charts

AG Charts is een krachtige JavaScript-bibliotheek voor het maken van interactieve en aanpasbare grafieken in webapplicaties. In het project wordt AG Charts gebruikt om visuele statistieken, zoals staafdiagrammen, weer te geven binnen de frontend. Dit helpt beheerders om snel inzicht te krijgen in het gebruik van de uitleenbibliotheek. (AG Charts, n.d.)

2.1.11. Angular

Voor de frontend werd gekozen voor Angular, versie 21, met gebruik van het nieuwe signals-model voor state management. Angular biedt een robuust framework voor het bouwen van schaalbare en onderhoudbare webapplicaties. De signals-functionaliteit zorgt voor efficiënte, reactieve UI-updates en een betere ontwikkelaarservaring. Dankzij Angular konden we snel een moderne, toegankelijke en performante SPA bouwen die eenvoudig integreert met de backend-API. (Angular, n.d.)

2.1.12. Ngx-translate

Voor het realiseren van een meertalige gebruikersinterface binnen het Angular-framework is gebruikgemaakt van Ngx-translate. Dit is een veelzijdige en breed geaccepteerde bibliotheek voor internationale (i18n) ondersteuning in Single Page Applications. De tool maakt het mogelijk om vertaalbestanden in JSON-formaat dynamisch in te laden, zonder dat de applicatie hiervoor opnieuw opgestart of gecompileerd hoeft te worden. (Ngx-translate, n.d.)

2.2. Packages

Packages zijn externe softwarebibliotheken die aan het project zijn toegevoegd om specifieke functionaliteit te bieden zonder deze zelf te hoeven ontwikkelen. Hieronder worden de belangrijkste gebruikte packages en hun rol binnen het project toegelicht.

2.2.1. Excel Data Reader -> ClosedXML

Aanvankelijk werd Excel Data Reader gebruikt, een .NET-bibliotheek waarmee Excel-bestanden (.xls, .xlsx) eenvoudig konden worden ingelezen en verwerkt. Dit was handig voor het importeren van LEGO-set gegevens uit Excel-bestanden, zodat beheerders snel grote hoeveelheden data konden toevoegen aan het systeem.

Naarmate het project vorderde, werd overgestapt op ClosedXML. Deze bibliotheek biedt niet alleen dezelfde leesfunctionaliteit, maar maakt het ook mogelijk om zelf Excel-werkbladen te genereren en te exporteren. Dit bleek essentieel voor het automatisch aanmaken van rapporten en exportbestanden, waardoor beheerders eenvoudig data uit het systeem kunnen halen voor verdere verwerking of rapportage.

2.2.2. QRCoder

QRCoder werd gebruikt voor het genereren van QR-codes binnen de applicatie. Deze QR-codes werden onder andere toegepast voor het identificeren van bedden in de uitleenbibliotheek. Door elk bed te voorzien van een unieke QR-code, werd het uitleenproces efficiënter en foutlozer, omdat gebruikers en beheerders snel en eenvoudig bedden konden scannen en identificeren.

2.2.3. Tailwind

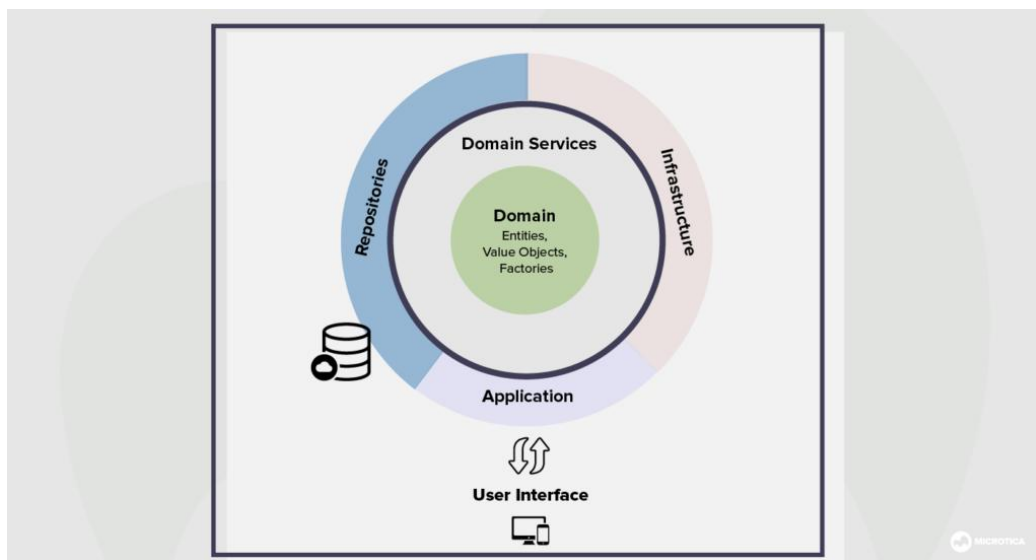
Tailwind CSS werd gebruikt als utility-first CSS framework voor het snel en consistent stylen van de frontend. Dankzij Tailwind konden we snel responsieve en moderne gebruikersinterfaces bouwen, zonder telkens custom CSS te moeten schrijven. Dit zorgde voor een uniforme look & feel doorheen de applicatie en versnelde het ontwikkelproces aanzienlijk.

2.3. Gebruikte structuur

De architectuur van het project is zorgvuldig opgezet om schaalbaarheid, onderhoudbaarheid en duidelijke domeinafspraken te waarborgen. Hieronder volgt een toelichting op de gehanteerde structuren.

2.3.1. Domain-Driven Design (DDD)

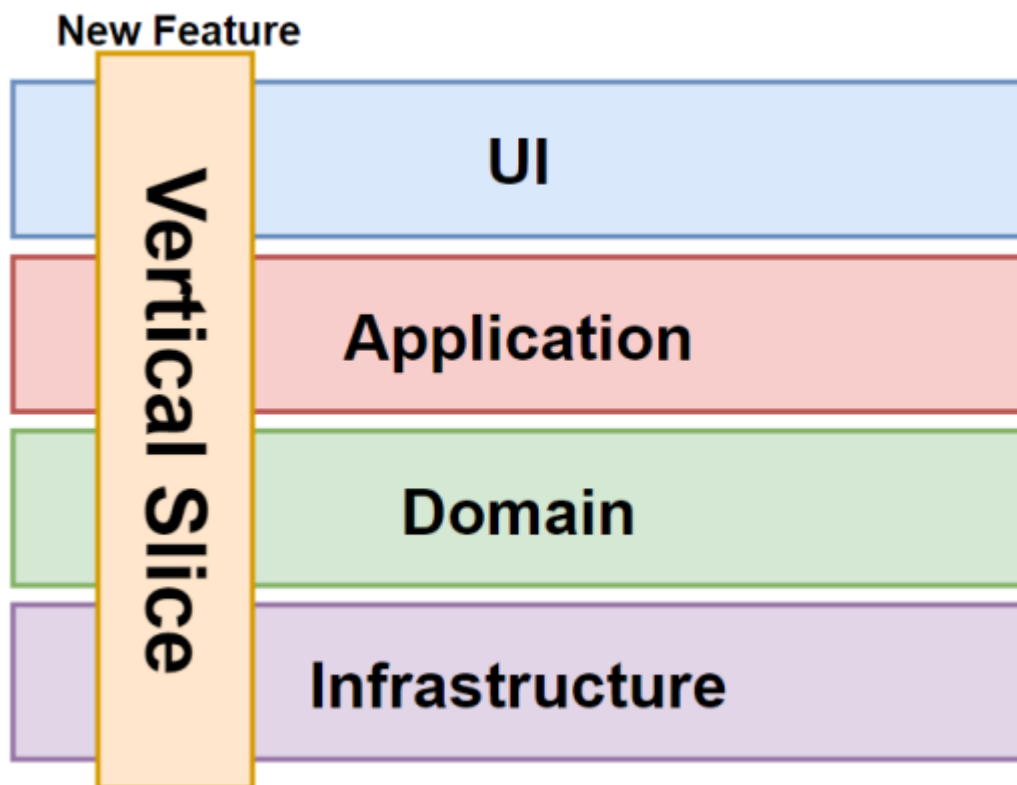
Binnen het project is Domain-Driven Design toegepast om de complexiteit van het domein te beheersen. Door het domein centraal te stellen in het ontwerp, werd een duidelijke scheiding tussen domeinlogica en infrastructuur gerealiseerd. Dit zorgt ervoor dat de kernlogica van de uitleenbibliotheek helder, goed testbaar en eenvoudig uitbreidbaar blijft, zelfs naarmate het systeem groeit.



Afbeelding 1: Domain-Driven Design

2.3.2. Vertical slice architecture

De Vertical Slice Architecture werd gehanteerd om functionaliteit op te delen in verticale sneden, waarbij elke slice een volledige functionele eenheid vormt (van API-endpoint tot database). Dit bevordert de modulariteit en maakt het eenvoudiger om nieuwe functionaliteit toe te voegen of bestaande onderdelen te wijzigen zonder impact op andere delen van het systeem. Hierdoor blijft het project overzichtelijk en schaalbaar.



Afbeelding 2: Vertical slice

2.3.3. Directives (Angular)

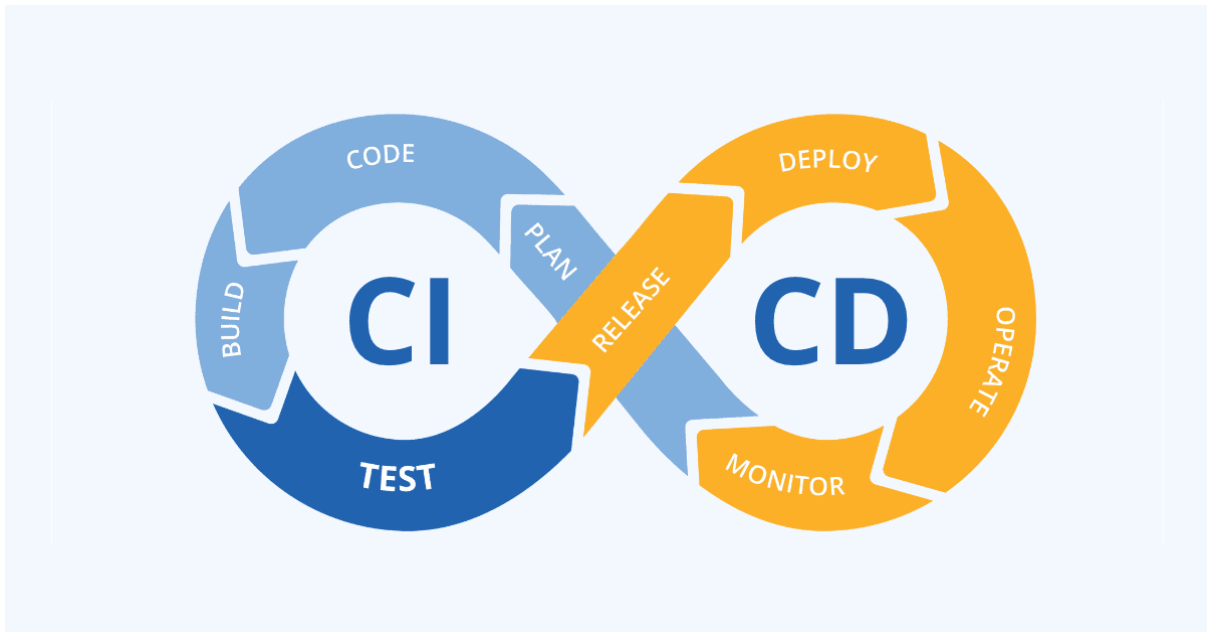
Directives in Angular zijn herbruikbare stukjes logica waarmee je het gedrag of de weergave van HTML-elementen kunt aanpassen. In het project worden directives ingezet om standaard stijlen en gedrag toe te voegen aan formulervelden en andere UI-elementen. Dit zorgt voor een consistente gebruikerservaring en maakt het eenvoudig om wijzigingen centraal door te voeren.

2.3.4. Github issues

Github Issues werden gebruikt voor het plannen, opvolgen en documenteren van taken, bugs en nieuwe features. Dit zorgde voor transparantie binnen het team en maakte het mogelijk om de voortgang van het project gestructureerd te beheren. Door issues te koppelen aan pull requests werd het reviewproces gestroomlijnd.

2.3.5. Geautomatiseerde ci/cd

Geautomatiseerde CI/CD (Continuous Integration/Continuous Deployment) werd opgezet via Github Actions. Hierdoor werden codewijzigingen automatisch getest en, na goedkeuring, uitgerold naar de test- of productieomgeving. Dit minimaliseerde fouten, versnelde het releaseproces en zorgde voor een betrouwbare oplevering van nieuwe functionaliteit.



Afbeelding 3: ci/cd pipeline

2.3.6. Signals (Angular)

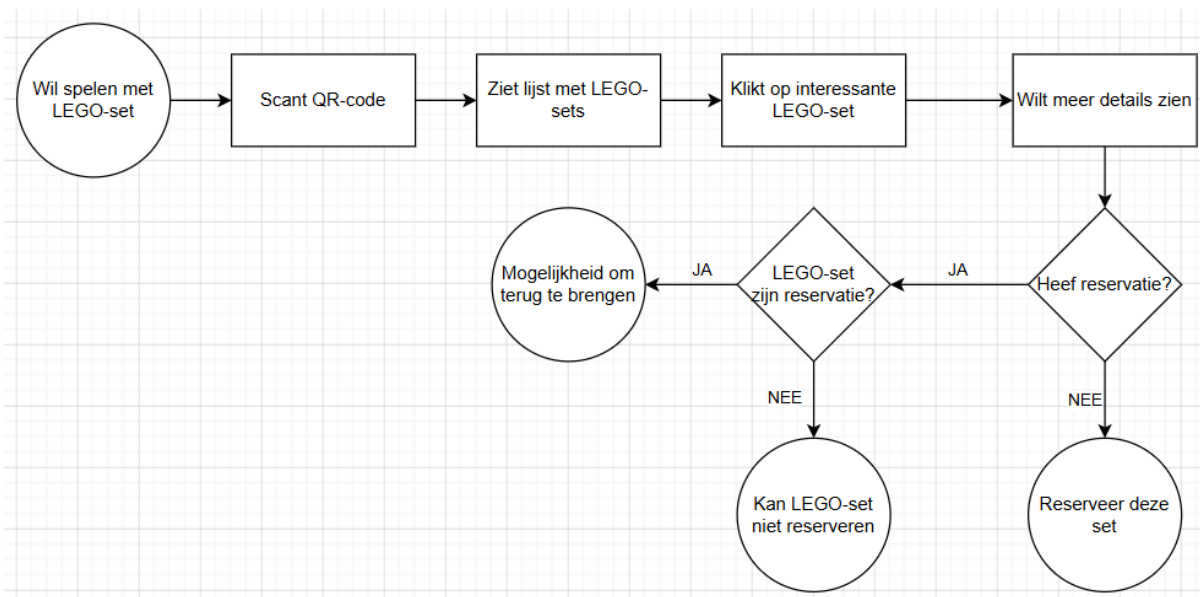
Binnen de frontend werd gebruikgemaakt van Angular signals voor state management. Signals bieden een efficiënte en reactieve manier om UI-updates te beheren, zonder de complexiteit van traditionele state management libraries. Dit resulteerde in een snellere, meer responsieve applicatie en een eenvoudigere codebase.

2.4. Gebruikers

Dit hoofdstuk zal beschrijven wie er allemaal in contact komt met de applicatie en hoe ze het gaan gebruiken. In het algemeen werken we met 3 portalen (reservatie-/uitleen-portaal, bibliotheekportaal en managementportaal). Elk portaal is bestemd voor een gebruiker.

2.4.1. Kinderen en Ouders (Reservatie-/Uitleenportaal)

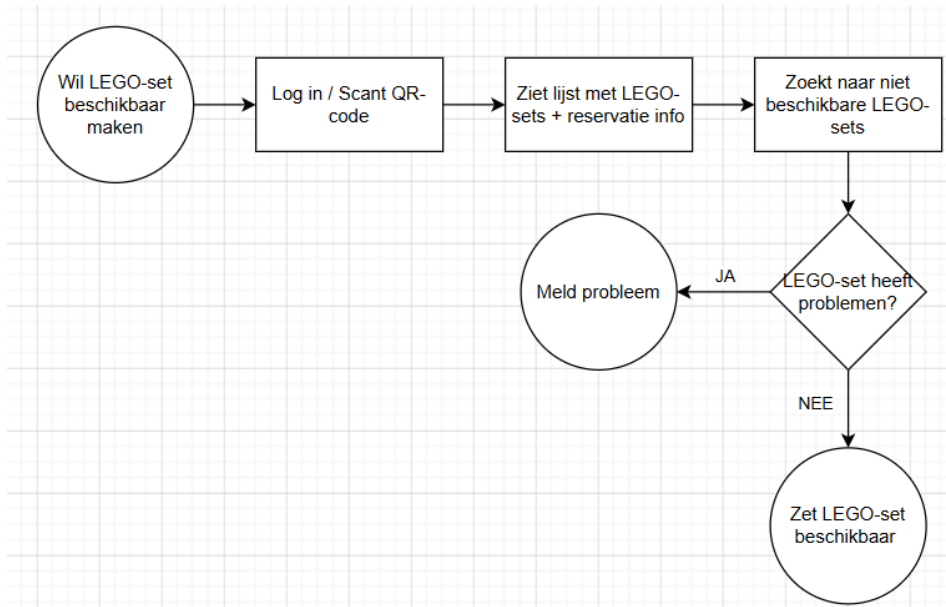
Kinderen die in het ziekenhuis verblijven, en hun ouders, gebruiken het reservatie- of uitleenportaal. Via dit portaal kunnen zij eenvoudig LEGO-sets reserveren die beschikbaar zijn in hun ziekenhuisbibliotheek. Ze kunnen het aanbod bekijken, sets selecteren en reserveren. Het portaal is gebruiksvriendelijk en visueel aantrekkelijk, zodat ook jonge kinderen zelfstandig of samen met hun ouders een keuze kunnen maken.



Afbeelding 4: Flow reservatie maken

2.4.2. Ziekenhuispersoneel (Bibliotheekportaal)

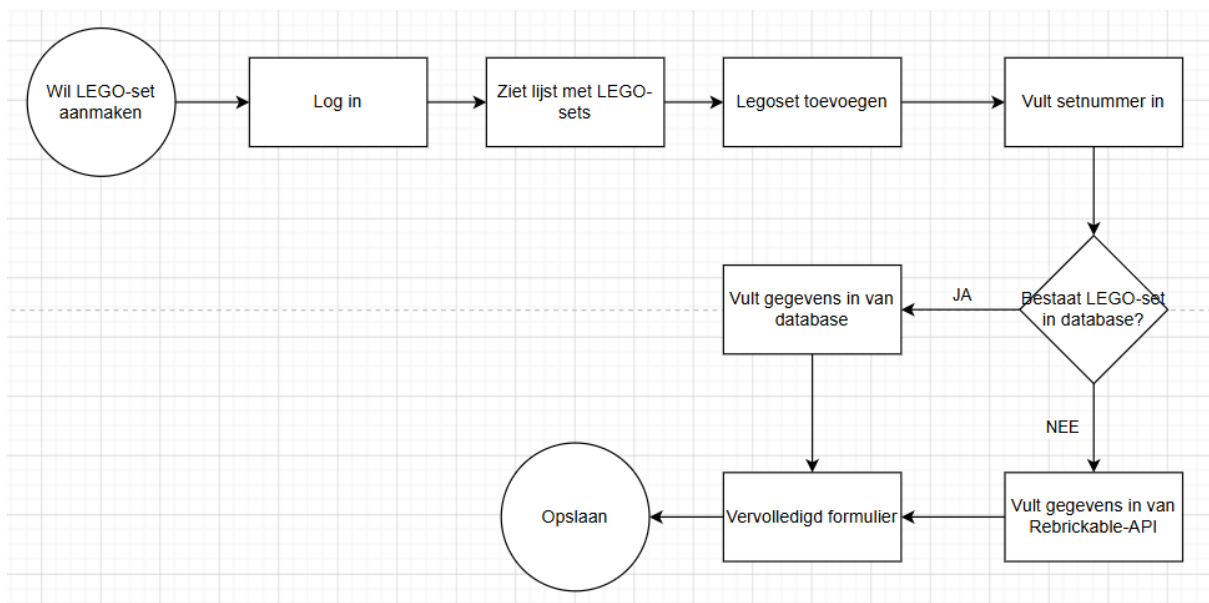
Het ziekenhuispersoneel gebruikt het bibliotheekportaal om het uitleenproces te beheren. Zij kunnen sets uitlenen en innemen, en de status van de inventaris opvolgen. Via het portaal kunnen ze ook problemen melden (zoals ontbrekende steentjes) en zien welke bedden welke sets in gebruik hebben. Dit portaal ondersteunt het personeel bij een vlot en foutloos beheer van de uitleenbibliotheek.



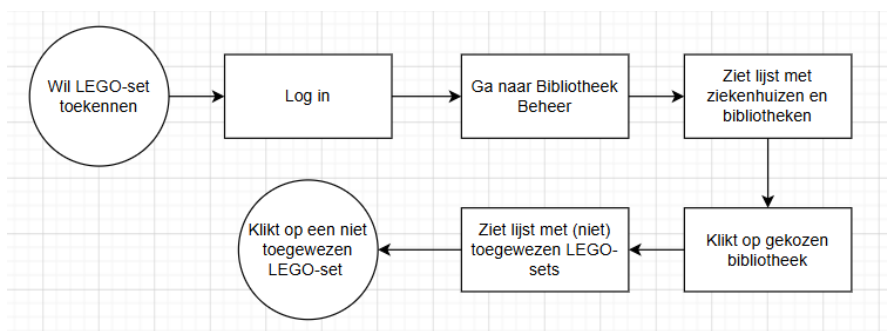
Afbeelding 5: Flow LEGO-set beschikbaar maken

2.4.3. Bricks and More (Beheerders, Managementportaal)

De beheerders van Bricks and More hebben toegang tot het managementportaal. Hier kunnen zij het volledige systeem beheren: nieuwe ziekenhuizen en bibliotheken toevoegen, het aanbod van LEGO-sets beheren, statistieken raadplegen, en gebruikersrechten toekennen. Dit portaal biedt diepgaand inzicht in het gebruik van de uitleenbibliotheken en ondersteunt strategische beslissingen, zoals het uitbreiden van het aanbod of het optimaliseren van de logistiek.



Afbeelding 6: Flow LEGO-set aanmaken



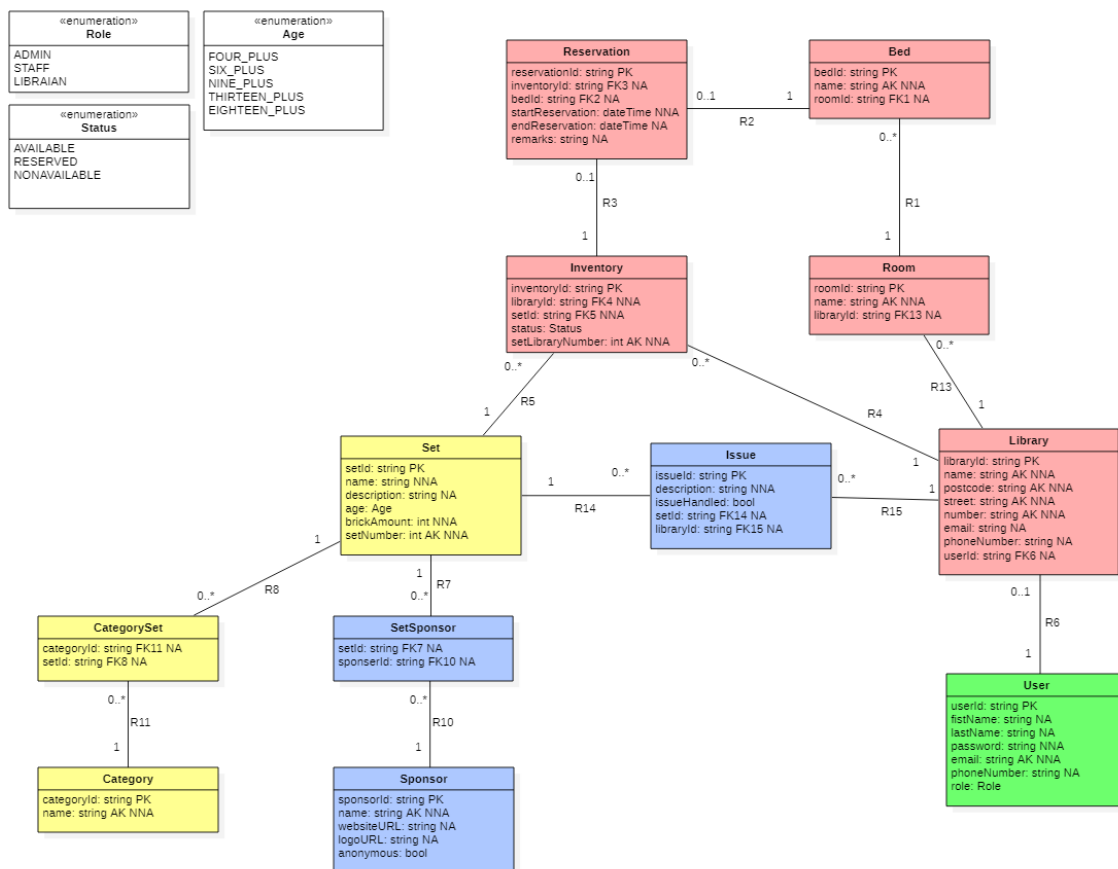
Afbeelding 7: Flow LEGO-set toekennen

3. Realisatie

Dit hoofdstuk beschrijft de realisatiefase van de verschillende systeemonderdelen, inclusief reflecties op de technische uitdagingen, de gemaakte keuzes en hun meerwaarde voor het project.

3.1. Ontwerp van de databankstructuur

De initiële fase van de realisatie bestond uit het opstellen van een Entity-Relationship Diagram (ERD) om de beoogde datastructuur te visualiseren. Dit diagram bood een essentieel overzicht van de complexe relaties tussen de verschillende entiteiten binnen het systeem, zoals ziekenhuizen, bibliotheken, bedden, LEGO-sets en reservaties. Door vooraf deze structuur uit te werken, werd het mogelijk om een schaalbare en toekomstbestendige database op te zetten. Dit voorkomt latere problemen bij uitbreidingen.



Afbeelding 8: Zelfgemaakte ERD

3.2. Integratie van de Rebrickable API

De koppeling met de Rebrickable API werd in een vroeg stadium van het project gerealiseerd. Het doel hiervan was om steeds actuele en gedetailleerde informatie over LEGO-sets te kunnen aanbieden, zonder deze handmatig te moeten invoeren. Na analyse en feedback werd de integratie geoptimaliseerd, zodat gegevensconsistentie gegarandeerd blijft en ongewenste overschrijvingen van data worden voorkomen. Dit verhoogt de betrouwbaarheid van de applicatie en vermindert de kans op fouten bij het beheren van het aanbod.



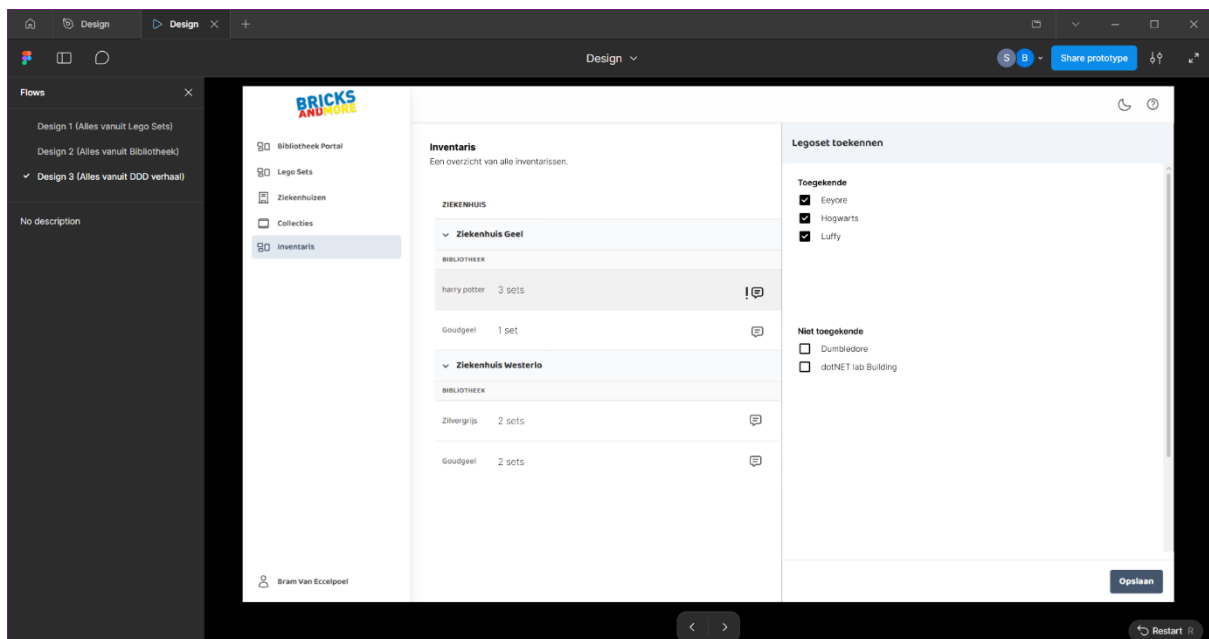
Afbeelding 9: Start van Rebrickable API

3.3. Implementatie van Domain-Driven Design

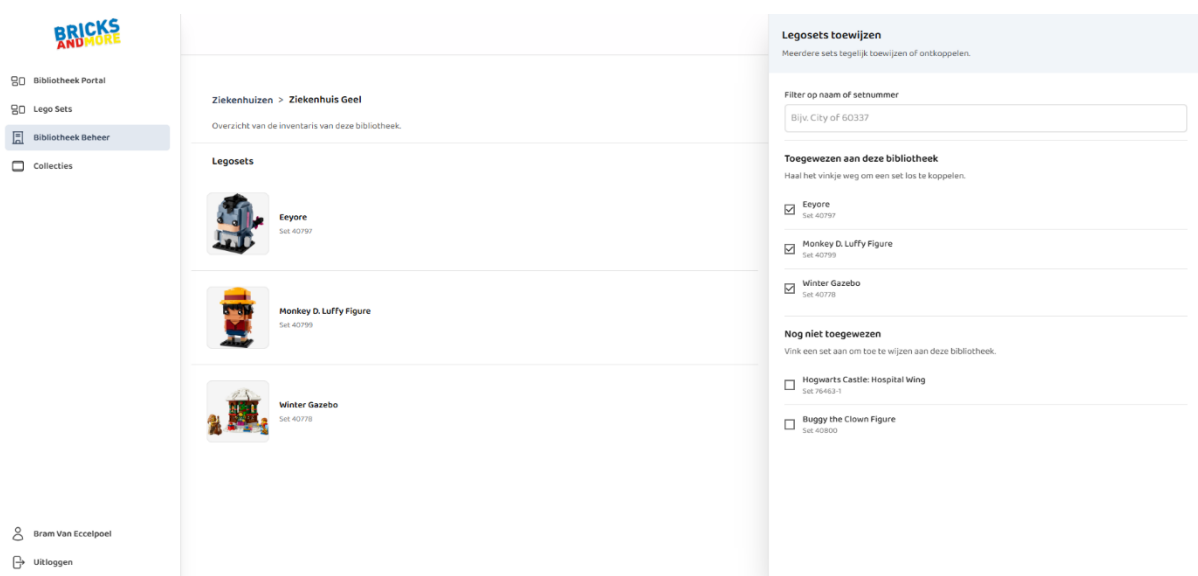
Zowel de backend als de frontend zijn herwerkt volgens de principes van Domain-Driven Design (DDD). Door het domein centraal te stellen, wordt de logica van het uitleenproces duidelijk gescheiden van technische details zoals opslag of communicatie. Dit maakt de code beter begripbaar, eenvoudiger te testen en makkelijker aan te passen bij veranderende eisen. De meerwaarde hiervan is dat het systeem duurzaam en onderhoudbaar blijft, ook op lange termijn.

3.4. Beheer van de bibliotheekinfrastructuur

Het systeem faciliteert het volledige beheer van de bibliotheekinfrastructuur via zowel de backend als de frontend. Dit omvat onder meer de toekenning van bibliotheken aan specifieke ziekenhuizen, het koppelen van LEGO-sets en gebruikers aan een bibliotheek, en het beheren van de inventaris. Door deze processen te digitaliseren, wordt het beheer veel efficiënter en minder foutgevoelig dan bij handmatige of Excel-gebaseerde oplossingen.



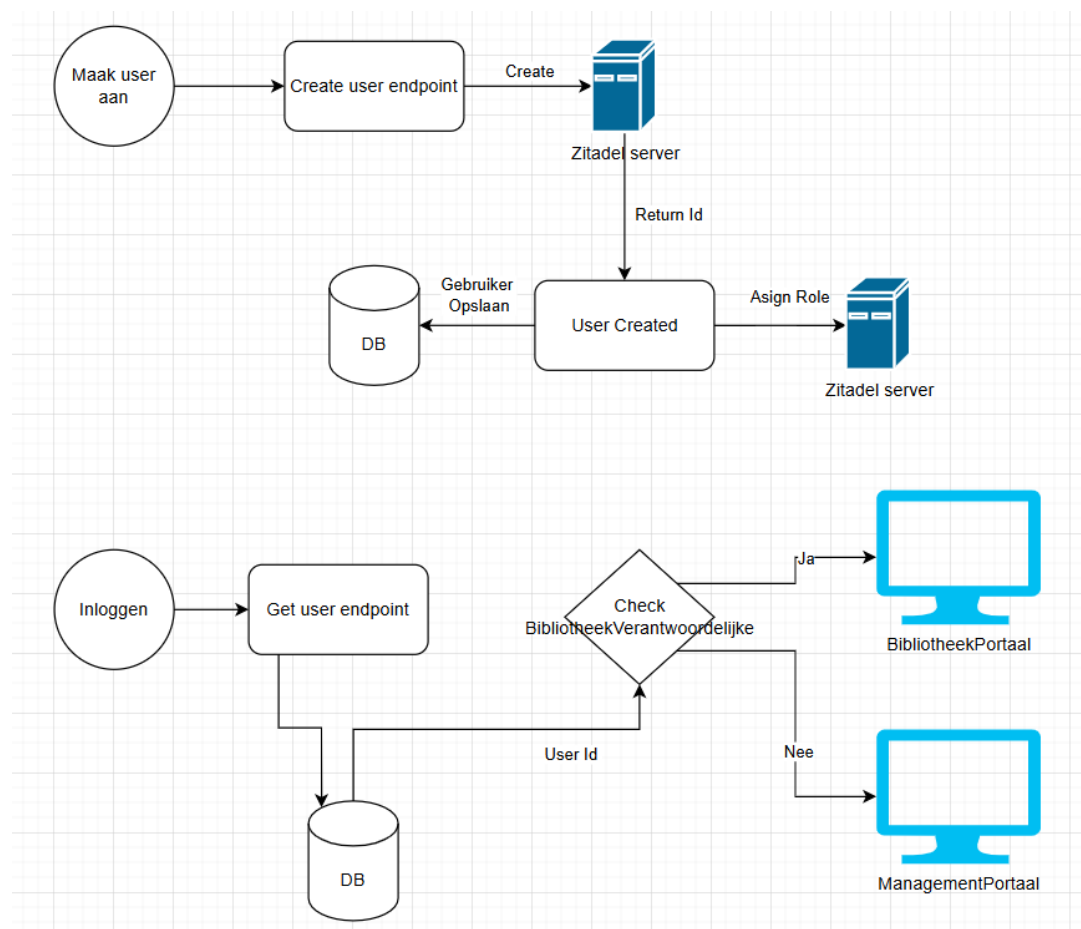
Afbeelding 10: Beheren (toekennen van een LEGO-set design)



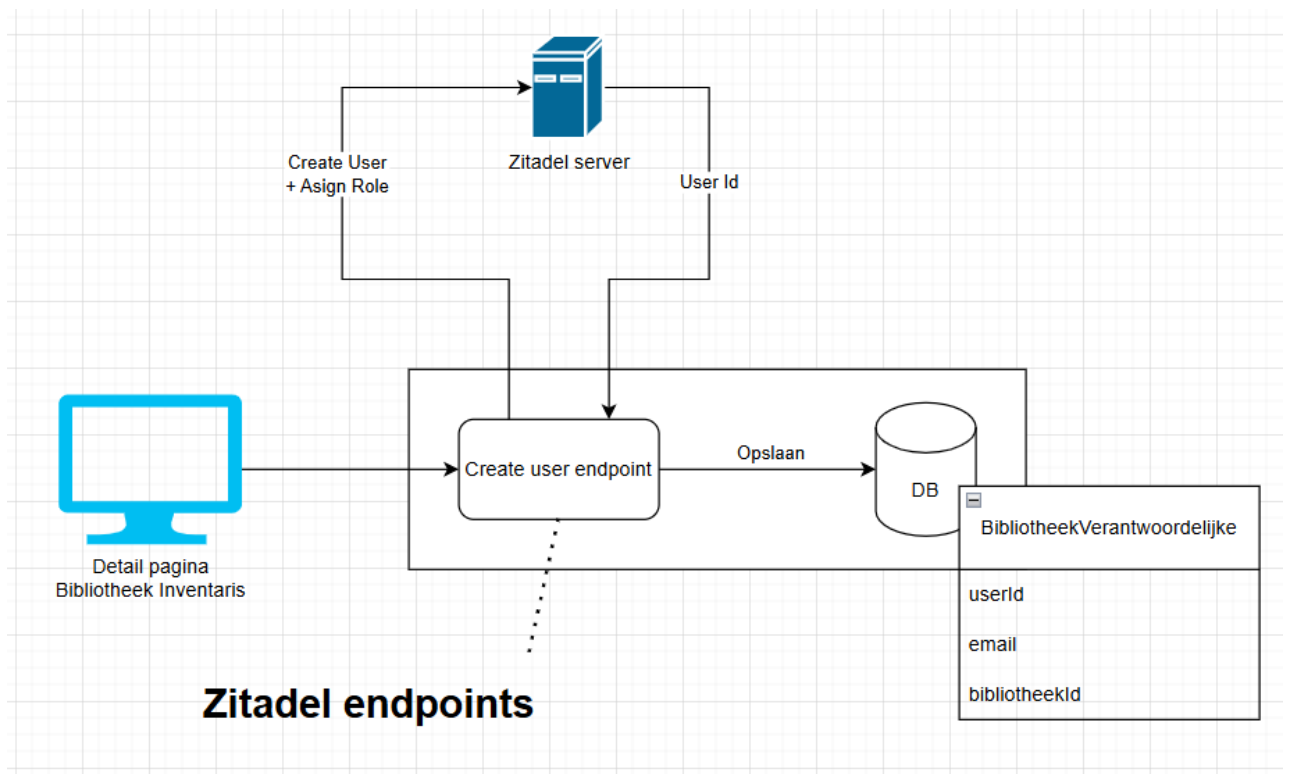
Afbeelding 11: Beheren (toekennen van een LEGO-set)

3.5. Implementatie van Zitadel en beveiliging

Zitadel fungeert binnen het project als de centrale oplossing voor identity- en access management. Het beveiligen van de API-endpoints en het correct toewijzen van gebruikersrollen zorgt ervoor dat elke gebruiker enkel toegang heeft tot de functionaliteiten die voor hem of haar bedoeld zijn. Dit verhoogt de veiligheid en het vertrouwen in het systeem, en voorkomt dat gevoelige gegevens of beheerfuncties per ongeluk toegankelijk zijn voor onbevoegden.



Afbeelding 12: Userflow aanmaken/inloggen van een gebruiker



Afbeelding 13: Backendflow van het aanmaken van een gebruiker

3.6. Ontwikkeling van het reservatieportaal

Het ontwerp en de realisatie van het reservatieportaal volgden een iteratief proces, waarbij gebruikersfeedback centraal stond. Hierdoor is een portaal ontstaan dat niet alleen gebruiksvriendelijk is, maar ook perfect aansluit bij de noden van kinderen, ouders en ziekenhuispersoneel. De meerwaarde hiervan is dat de drempel om LEGO-sets te reserveren laag is, wat het gebruik en de tevredenheid verhoogt.



Afbeelding 14: Reservatieportaal design (oude)



Afbeelding 15: Reservatieportaal design (officiële)

3.7. Beveiliging van API-endpoints

Het waarborgen van een sluitende beveiliging voor de API-endpoints was een prioritair aandachtspunt. Door endpoints te autoriseren op basis van gebruikersrollen, wordt de toegang tot kritieke functionaliteiten strikt beperkt tot geautoriseerde gebruikersgroepen. Dit beschermt de integriteit en vertrouwelijkheid van de data en voldoet aan de gestelde veiligheidseisen.

3.8. Optimalisatie van de QR-code functionaliteit

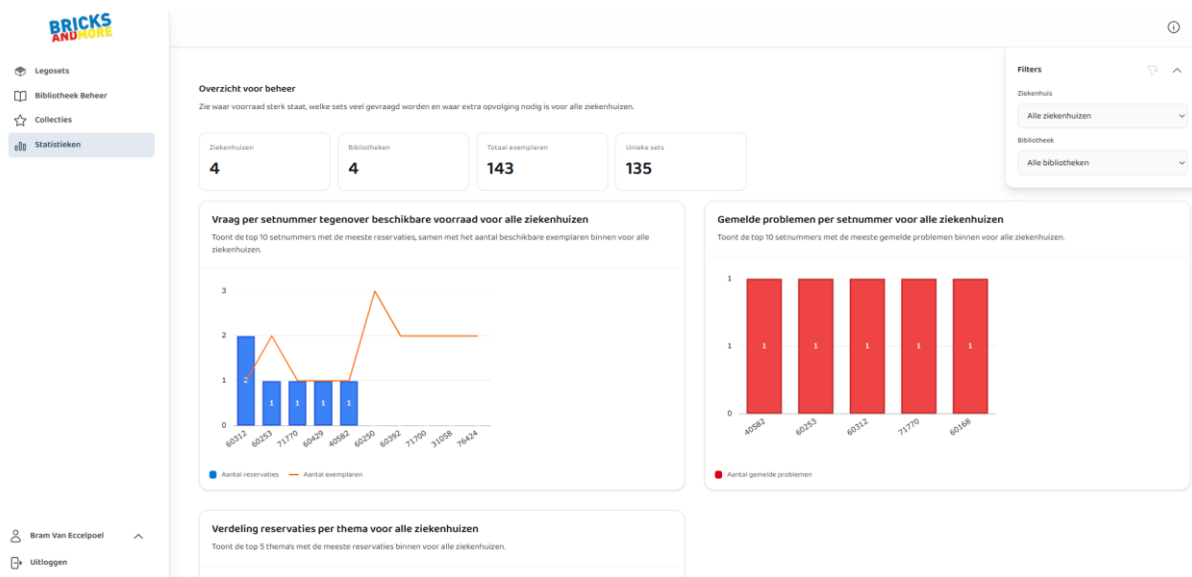
QR-codes werden ingezet om het uitleenproces te vereenvoudigen en te versnellen. Door elk bed en elke set te voorzien van een unieke QR-code, kunnen gebruikers en personeel snel en foutloos identificeren en registreren. De continue optimalisatie van deze functionaliteit heeft geleid tot een soepele gebruikerservaring en een efficiënter proces.



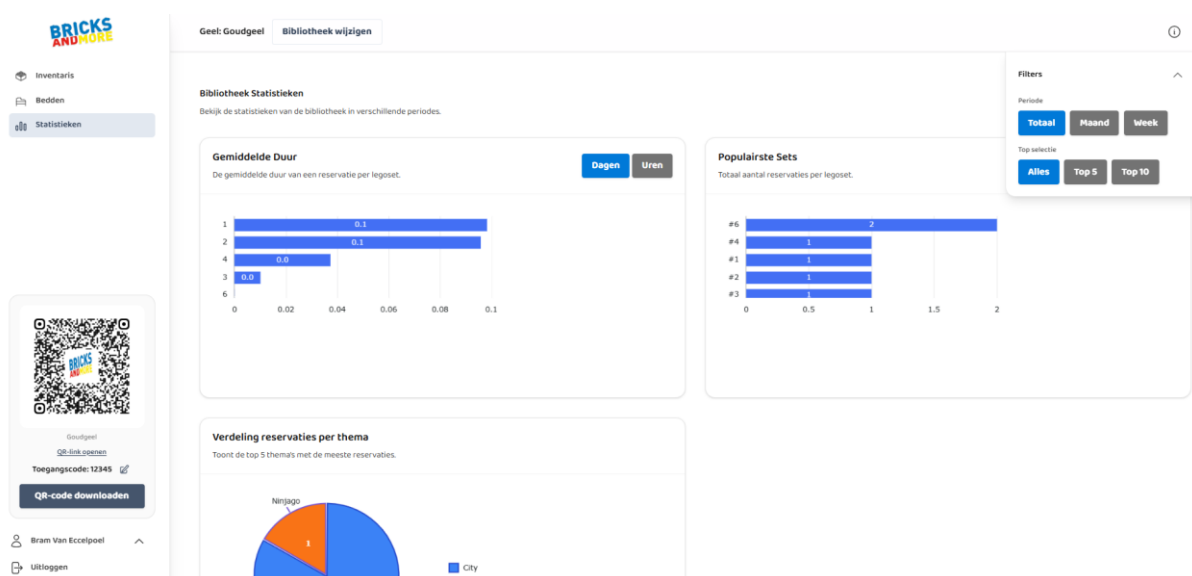
Afbeelding 16: Voorbeeld QR-code

3.9. Implementatie van statistiekpagina's

Voor ziekenhuispersoneel en beheerders zijn statistiekpagina's ontwikkeld die inzicht geven in het gebruik van de uitleenbibliotheek. Door gebruik te maken van AG Charts worden data visueel en overzichtelijk gepresenteerd. Dit ondersteunt het nemen van onderbouwde beslissingen, zoals het uitbreiden van het aanbod of het optimaliseren van de logistiek.



Afbeelding 17: Statistieken pagina managementportaal



Afbeelding 18: Statistieken pagina bibliotheekportaal

3.10. Importfunctionaliteit via Excel-bestanden

Om de overgang naar een digitale oplossing te vergemakkelijken, is een importfunctionaliteit ontwikkeld waarmee bestaande gegevens uit Excel-bestanden in bulk kunnen worden toegevoegd. Dit bespaart veel tijd en voorkomt fouten die bij handmatige invoer zouden ontstaan. Het maakt het ook mogelijk om snel te starten met de applicatie, zelfs als er al een bestaande administratie is.

3.11. Beheer en weergave van collecties

De datastructuur voor collecties is functioneel geactiveerd, zodat LEGO-sets nu aan een specifieke collectie kunnen worden gekoppeld. Dit zorgt voor een gestructureerd overzicht in het reservatieportaal en maakt het voor gebruikers eenvoudiger om sets te vinden die bij een collectie toebehoren. Zij kunnen de informatie over de uitgever van de collectie ook raadplegen.



Afbeelding 19: Collectie op een LEGO-set

3.12. Exportfunctionaliteit naar Excel-bestanden

Om beheerders en personeel te ondersteunen bij het maken van etiketten voor op de LEGO-dozen, is er een exportmogelijkheid voorzien waarmee de data van de legosets eenvoudig naar een Excel-bestand kan worden geëxporteerd. Dit bestand kan vervolgens direct gebruikt worden om etiketten aan te maken, zonder dat gebruikers zelf handmatig een Excel-overzicht hoeven samen te stellen. Deze functionaliteit bespaart tijd, voorkomt fouten en zorgt voor een uniforme en efficiënte werkwijze bij het labelen van de LEGO-sets.

3.13. Implementatie van meerdere talen

Om de toegankelijkheid voor kinderen, ouders en zorgpersoneel te waarborgen, is de applicatie volledig meertalig (Nederlands en Engels) opgezet met *Ngx-translate*. De vertalingen worden centraal beheerd via gestructureerde JSON-bestanden (nl.json en en.json).

De grootste technische uitdaging lag in de state-reactiviteit en het behoud van de gebruikersvoorkeur. Dit is opgelost door de geselecteerde taal op te slaan in de localStorage, waardoor de applicatie bij een volgend bezoek automatisch in de juiste taal opstart.



Afbeelding 20: Dropdown talen

3.14. Ontwikkeling van de landingspagina

Om gebruikers direct een warm welkom te heten, is er een vrolijke en uitnodigende landingspagina ontworpen. Dit vormt de vertrouwde basis van de applicatie waar iedereen intuïtief zijn weg kan vinden. Bij het ontwerp stond een fijne gebruikerservaring centraal, zodat de pagina niet alleen visueel aantrekkelijk is, maar ook direct zorgt voor rust en duidelijkheid. Het resultaat is een helder startpunt dat uitnodigt tot interactie en iedereen moeiteloos naar de juiste plek binnen de applicatie leidt.



Afbeelding 21: Landingspagina

4. Besluit

De ontwikkeling van deze softwareoplossing voor het beheer en de uitleen van LEGO-sets in ziekenhuizen was een bijzonder leerrijke en boeiende ervaring. Het project bracht verschillende technische uitdagingen met zich mee, maar door het gebruik van moderne tools, doordachte architecturale keuzes en een iteratieve aanpak zijn deze telkens succesvol overwonnen.

De samenwerking binnen het team, het regelmatig inwinnen van feedback en het openstaan voor suggesties hebben een cruciale rol gespeeld in het eindresultaat. Door constructief samen te werken en elkaars expertise te benutten, werd niet alleen de kwaliteit van de applicatie verhoogd, maar groeide ook het wederzijds vertrouwen en de teamspirit.

Het eindproduct is een innovatieve, schaalbare en gebruiksvriendelijke applicatie die een duidelijke meerwaarde biedt voor zowel ziekenhuizen, beheerders als eindgebruikers. De digitalisering van het uitleenproces zorgt voor een efficiëntere werking, minder fouten en een betere gebruikerservaring. De applicatie is bovendien flexibel genoeg om in de toekomst verder uit te breiden.

Terugkijkend op het traject kan gesteld worden dat het doel (het zelfstandig realiseren van een complex softwareproject volgens de geldende best practices) ruimschoots werd bereikt. De opgedane kennis, de samenwerking en het enthousiasme binnen het team maken dit project tot een waardevolle en geslaagde ervaring.

Gebruik van Artificial Intelligence (AI)

Bij de totstandkoming van dit reflectiedocument is gebruikgemaakt van Artificial Intelligence (AI). De AI-ondersteuning is specifiek ingezet als redactioneel hulpmiddel voor het controleren en verbeteren van de spelling, grammatica en zinsbouw. Hierdoor is de inhoudelijke essentie van mijn stage-ervaringen volledig behouden, terwijl de tekstuele kwaliteit en professionaliteit naar een hoger niveau zijn getild.

LITERATUURLIJST

Onderstaande literatuurlijst biedt een volledig overzicht van de bronnen. Hierbij is een onderscheid gemaakt tussen de geraadpleegde literatuur en gesproken bronnen, welke gezamenlijk hebben gediend als fundament voor mijn onderzoek.

4.1. Gesproken bronnen

- **Deschryver Tim** (Mentor): Begeleiding bij complexe technische vraagstukken.
- **Wuyts Geoffrey**: Advies met betrekking tot het visuele aspect en de gebruiksvriendelijkheid van de applicatie.
- **Feedbackgroep**: Ive Verstappen, Daan Geudens, Bjorn Ruysen, Katleen Vanhees en Pieter Van Aerde (tijdens demo-sessies).

4.2. Geschreven bronnen

- **AG Charts Documentatie** (AG Charts, n.d.)
- **Deschryver, T.**: *Refactor your .NET HTTP clients to typed HTTP clients*. (Tim Deschryver Blog (Typed Clients), n.d.)
- **Pluralsight**: Diverse cursussen over Domain-Driven Design. (Pluralsight, n.d.)
- **Rebrickable API Documentatie** (Rebrickable, n.d.)
- **Zitadel Documentatie** (Zitadel, n.d.)

BIJLAGEN

- (n.d.). Retrieved from Docker: <https://www.docker.com>
- (n.d.). Retrieved from Visual Studio Code: <https://code.visualstudio.com>
- (n.d.). Retrieved from Figma: <https://www.figma.com>
- (n.d.). Retrieved from StarUML: <https://staruml.io>
- (n.d.). Retrieved from Zitadel: <https://zitadel.com>
- (n.d.). Retrieved from Rebrickable: <https://rebrickable.com>
- (n.d.). Retrieved from AG Charts: <https://ag-grid.com/charts/>
- (n.d.). Retrieved from PostgreSQL: <https://www.postgresql.org>
- (n.d.). Retrieved from Pluralsight: <https://app.pluralsight.com/library/>
- (n.d.). Retrieved from Tim Deschryver Blog (Typed Clients):
<https://timdeschryver.dev/blog/refactor-your-net-http-clients-to-typed-http-clients>
- (n.d.). Retrieved from Dokploy: <https://dokploy.com>
- (n.d.). Retrieved from Angular: <https://angular.dev>
- (n.d.). Retrieved from Github: <https://github.com>
- (n.d.). Retrieved from Ngx-translate: <https://ngx-translate.org>

Afbeelding 1: Domain-Driven Design	9
Afbeelding 2: Vertical slice	10
Afbeelding 3: ci/cd pipeline	12
Afbeelding 4: Flow reservatie maken	13
Afbeelding 5: Flow LEGO-set beschikbaar maken	14
Afbeelding 6: Flow LEGO-set aanmaken.....	15
Afbeelding 7: Flow LEGO-set toekennen.....	15
Afbeelding 8: Zelfgemaakte ERD	16
Afbeelding 9: Start van Rebrickable API	17
Afbeelding 10: Beheren (toekennen van een LEGO-set design)	19
Afbeelding 11: Beheren (toekennen van een LEGO-set)	19
Afbeelding 12: Userflow aanmaken/inloggen van een gebruiker	20
Afbeelding 13: Backendflow van het aanmaken van een gebruiker.....	21
Afbeelding 14: Reservatieportaal design (oude).....	22
Afbeelding 15: Reservatieportaal design (officiële)	22
Afbeelding 16: Voorbeeld QR-code	23
Afbeelding 17: Statistieken pagina managementportaal	24
Afbeelding 18: Statistieken pagina bibliotheekportaal	24
Afbeelding 19: Collectie op een LEGO-set	25
Afbeelding 20: Dropdown talen	26
Afbeelding 21: Langdingspagina	27